

FC7xxx MSC User Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/10/20	Initial release for MCAL V0.3.0

Table of Contents

Chapter 1 MSC Introduction	5
1.1 Requirements.....	5
1.2 Design Summary	5
1.3 Hardware Summary.....	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure.....	6
2.3 Macros	6
2.3.1 CDD_Msc.h.....	6
2.3.2 CDD_Msc_Cfg.h	8
2.3.3 CDD_Msc_PBCfg.h	9
2.4 Enums.....	9
2.4.1 Enums in CDD_Msc_Cfg.h.....	9
2.4.2 Enums in CDD_Msc.h	11
2.5 Structures.....	13
2.5.1 Msc_TransmissionSourceConfigType	13
2.5.2 Msc_ChannelConfigType.....	13
2.5.3 Msc_ConfigType	14
2.6 API Functions	14
2.6.1 Functions in CDD_Msc.h	14
2.7 Hardware Functions	19
2.7.1 Functions in CDD_Msc_LLD.h.....	19
2.8 Peripheral Functions	21
2.8.1 Functions in CDD_Msc_Hw.h	21
2.9 API Sequence Diagram	27
2.9.1 Frame Transmission	27
2.9.2 Frame Reception.....	28
Chapter 3 Tresos Configuration Items	30
3.1 Container Inclusion Relation.....	30
3.2 Containers and Variables	30
3.2.1 MscGeneral.....	30
3.2.2 MscGlobalConfig	33
3.2.3 CommonPublishedInformation	43
Chapter 4 Configuration Guides.....	47
4.1 MSC Usage Common Steps	47

4.2	MSC Channel Demo	47
-----	------------------------	----

Chapter 1 MSC Introduction

1.1 Requirements

The design of this module follows the specifications of the Msc driver specified in AUTOSAR Classic Platform Release 4.6.0.

1.2 Design Summary

The MSC is a dedicated serial interface designed specifically for connecting external power devices. Its serial data transfer capability reduces the number of pins required for such connections. Parallel data or command information is transmitted to the power device through a high-speed synchronous serial data stream (downstream channel). The MSC receives data and status from the power device through a low-rate asynchronous serial data stream (upstream channel). Sequential execution of parallel requests from the on-chip bus master to the module is facilitated by the on-chip bus system.

The MSC driver is a part of the microcontroller abstraction layer (MCAL), performing the hardware access and offers a hardware independent API to the upper layer. The only upper layer, which can access to the MSC driver, is the MSC Interface. The MSC driver can support more than one channel. This means that the MSC driver can handle one or more MSC channels as long as they belong to the same MSC hardware unit.

The MSC Driver for FC7300 uses MSC on-chip hardware modules which provides special support for the MSC protocol. The MSC physical interface should be connected to the MSC module pins.

1.3 Hardware Summary

There are two Micro Second Channel (MSC) interface instances for the FC7300. The acceptable maximum frequency of the MSC function clock is 100 MHz.

The feature set includes:

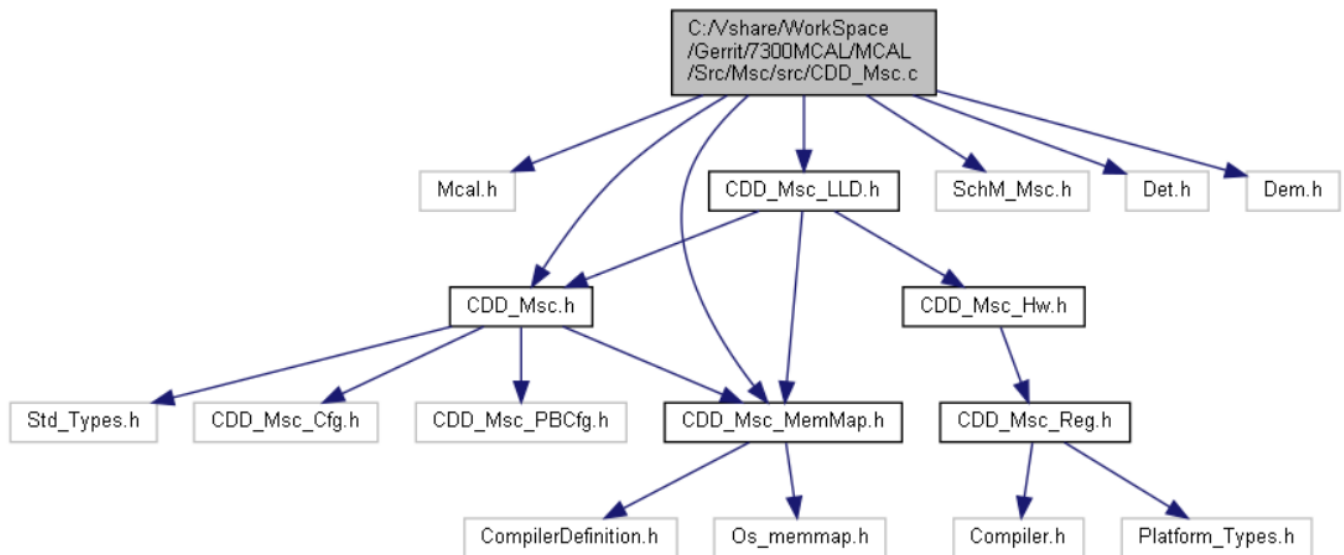
- Synchronous transmitting channel:
 - Baud rate: $f_{FCL} = f_{MSC}/2$
 - Three types of frames: command, data, and passive frame
 - Optional SEL bit in command and data frames
 - Emergency stop without CPU intervention
 - Trigger and repetition modes are supported
- Asynchronous receiving channel:
 - Baud rate: f_{MSC} divided by 4, 8, 16, 32, 64, 128, or 256
 - Error diagnosis including parity error and stop bit error
 - Spiker filter for asynchronous serial reception
 - 4-out-of-1 channel selector
 - Optional address field in each frame.

Chapter 2 Software Design

2.1 Rejected Requirements

N/A.

2.2 File Structure



2.3 Macros

2.3.1 CDD_Msc.h

- **#define MSC_VENDOR_ID 174**
MSC_VENDOR_ID definition.
- **#define MSC_MODULE_ID 253**
MSC_MODULE_ID definition.
- **#define MSC_AR_RELEASE_MAJOR_VERSION 4**
MSC_AR_RELEASE_MAJOR_VERSION definition.
- **#define MSC_AR_RELEASE_MINOR_VERSION 6**
MSC_AR_RELEASE_MINOR_VERSION definition.
- **#define MSC_AR_RELEASE_REVISION_VERSION 0**
MSC_AR_RELEASE_REVISION_VERSION definition.

- #define **MSC_SW_MAJOR_VERSION** 0
MSC_SW_MAJOR_VERSION definition.
- #define **MSC_SW_MINOR_VERSION** 1
MSC_SW_MINOR_VERSION definition.
- #define **MSC_SW_PATCH_VERSION** 0
SC_SW_PATCH_VERSION definition.
- #define **MSC_INSTANCE_ID** (15U)
MSC Instance ID.
- #define **MSC_E_PARAM_VALUE** ((uint8)0x00U)
API service called with a NULL pointer.
- #define **MSC_E_INVALID_POINTER** ((uint8)0x01U)
API service called with invalid configuration pointer.
- #define **MSC_E_UNINIT** ((uint8)0x03U)
API service used without module initialization.
- #define **MSC_E_STATE_TRANSITION** ((uint8)0x04U)
Invalid state transition for the current state.
- #define **MSC_E_CHANNEL_ACTIVE** ((uint8)0x05U)
API service called with a NULL pointer.
- #define **MSC_E_CHANNEL_NOT_STARTED** ((uint8)0x06U)
API service called with channel not started.
- #define **MSC_E_INVALID_CHANNEL** ((uint8)0x07U)
API service used with an invalid or inactive channel parameter.
- #define **MSC_E_CONFIG_VALUE** ((uint8)0x08U)
API service called in a not allowed configuration.
- #define **MSC_E_INVALID_RXINDEX** ((uint8)0x09U)
API service used with an invalid or inactive rx index parameter.
- #define **MSC_E_TX_BUSY** ((uint8)0x0AU)
API service used while tx channel is in invalid status.

- #define **MSC_E_RX_BUSY** ((uint8)0x0BU)
API service used while tx channel is in invalid status.
- #define **MSC_UNINIT** 0x01U
MSC driver states.
- #define **MSC_INIT** 0x02U
MSC driver states.
- #define **MSC_CHANNEL_ACTIVE** 0x03U
MSC channel states.
- #define **MSC_CHANNEL_IDLE** 0x04U
MSC channel states.

2.3.2 CDD_Msc_Cfg.h

- #define **MSC_PRECOMPILE_SUPPORT** (STD_ON)
MSC Precompile support.
- #define **MSC_HW_MAX_MODULES** 2U
No of Channels configured.
- #define **MSC_MAX_PARTITIONS** ((uint32)3U)
All CoreIDs are supported by MSC driver.
- #define **MSC_UNINIT_ARRAY** {MSC_UNINIT, MSC_UNINIT, MSC_UNINIT}
Msc uninit array.
- #define **MSC_CFG_PTR_INIT_ARRAY** {NULL_PTR, NULL_PTR, NULL_PTR}
Msc config init array.
- #define **MSC_POLARITY_IDENTICAL** (0U)
Msc signal polarity identical.
- #define **MSC_POLARITY_INVERTED** (1U)
Msc signal polarity interted.
- #define **MSC_TRANS_SOURCE_DATA_REG** 0x0U
SSL[x] is taken from TCDAR.DL[x].
- #define **MSC_TRANS_SOURCE_DATA_ALTIN** 0x2U
SSL[x] is taken from the ALTINL input line x.

- #define **MSC_TRANS_SOURCE_DATA_ALTIN_INV** 0x3U
SSL[x] is taken from the ALTINL input line x in inverted state.

2.3.3 CDD_Msc_PBCfg.h

- #define **MSC_DEV_ERROR_DETECT** (STD_ON)
Switches the Development Error Detection and Notification ON or OFF.
- #define **MSC_MULTICORE_SUPPORT** (STD_ON)
Switches the Development Error Detection and Notification ON or OFF.
- #define **MSC_VERSION_INFO_API** (STD_ON)
Support for version info API.
- #define **MSC_DISABLE_DEM_REPORT_ERROR_STATUS** (STD_OFF)
Disable Dem report.
- #define **MSC_TIMEOUT_LOOPS** 10000U
Number of loops before returning MSC_E_TIMEOUT.
- #define **MSC_E_TIMEOUT_VALUE** DemConf_DemEventParameter_DemEventParameter_MscTimeout
- #define **MSC_POLLING_USED** (STD_ON)
Msc data using polling.
- #define **MSC_GetCoreID()** GET_CPU_ID()
Get Core ID.

2.4 Enums

2.4.1 Enums in CDD_Msc_Cfg.h

2.4.1.1 Msc_TransmissionTriggerType

Enumeration	Msc_TransmissionTriggerType	
Description	MSC transmission type.	
Values	Value	Description
	MSC_TX_TRIGGER_SINGLE	Single Data Frame transfer selection (SW transmission).
	MSC_TX_TRIGGER_CONTINUOUS	Continuous Data Frame transfer selection (HW PWM transmission).

2.4.1.2 Msc_TransmissionPolarityType

Enumeration	Msc_TransmissionPolarityType
Description	MSC transmission polarity.

Values	Value	Description
	MSC_TX_NORMAL_POLARITY	Clock ChipSelect and Data are active high.
	MSC_TX_INVERTED_POLARITY_FSC	ChipSelect and Data are active high, Clock is active low.
	MSC_TX_INVERTED_POLARITY_EN	Clock and Data are active high, ChipSelect is active low.
	MSC_TX_INVERTED_POLARITY_SO	ChipSelect and Clock are active high, Data is active low.
	MSC_TX_INVERTED_POLARITY_FSC_EN	Data is active high, Clock and ChipSelect are active low.
	MSC_TX_INVERTED_POLARITY_FSC_SO	ChipSelect is active high, Clock and Data are active low.
	MSC_TX_INVERTED_POLARITY_EN_SO	Clock is active high, ChipSelect and Data are active low.
	MSC_TX_INVERTED_POLARITY_FSC_EN_SO	Clock ChipSelect and Data are active low.

2.4.1.3 Msc_FclActiveType

Enumeration	Msc_FclActiveType	
Description	MSC FCL active type.	
Values	Value	Description
	MSC_FCL_PARTITION_ACTIVE	Indicate that MSC channel clock is present only on downstream transmission.
	MSC_FCL_ALWAYS_ACTIVE	Indicates that MSC channel clock is always present.

2.4.1.4 MSC_ReceiveFrameType

Enumeration	MSC_ReceiveFrameType	
Description	MSC receive frame type.	
Values	Value	Description
	MSC_RX_12_BIT_LENGTH	12-bit frame selected.
	MSC_RX_16_BIT_LENGTH	16-bit frame selected.

2.4.1.5 Msc_RxDataParityType

Enumeration	Msc_RxDataParityType	
Description	MSC receive data parity.	
Values	Value	Description
	MSC_RX_PARITY_EVEN	Check for Even Parity.
	MSC_RX_PARITY_ODD	failure/error Std_Type return code.

2.4.1.6 MSC_ReceiveBaudDivType

Enumeration	MSC_ReceiveBaudDivType	
Description	MSC_receive baudrate division.	
Values	Value	Description
	MSC_RECEIVE_DISABLED	Receiving channel is disabled.
	MSC_BAUDRATE_FMSC_DIV4	Baud rate=fmsc/4.
	MSC_BAUDRATE_FMSC_DIV8	Baud rate=fmsc/8.
	MSC_BAUDRATE_FMSC_DIV16	Baud rate=fmsc/16.
	MSC_BAUDRATE_FMSC_DIV32	Baud rate=fmsc/32.

	MSC_BAUDRATE_FMESC_DIV64	Baud rate=fmesc/64.
	MSC_BAUDRATE_FMESC_DIV128	Baud rate=fmesc/128.
	MSC_BAUDRATE_FMESC_DIV256	Baud rate=fmesc/256.

2.4.1.7 Msc_ENxActiveType

Enumeration	Msc_ENxActiveType	
Description	MSC EN Selection	
Values	Value	Description
	MSC_EN_SELECTION_0	Select EN0.
	MSC_EN_SELECTION_1	Select EN1.
	MSC_EN_SELECTION_2	Select EN2.
	MSC_EN_SELECTION_3	Select EN3.

2.4.1.8 Msc_ENxType

Enumeration	Msc_ENxType	
Description	Msc EN Type.	
Values	Value	Description
	MSC_ENC	Select ENC.
	MSC_ENH	Select ENH.
	MSC_ENL	Select ENL.

2.4.1.9 MSC_SDISelectionType

Enumeration	MSC_SDISelectionType	
Description	MSC SDI selection.	
Values	Value	Description
	MSC_SDI_SEL_SDI0	SDI0 input is selected for SDI.
	MSC_SDI_SEL_SDI1	SDI1 input is selected for SDI.
	MSC_SDI_SEL_SDI2	SDI2 input is selected for SDI.
	MSC_SDI_SEL_SDI3	SDI3 input is selected for SDI.
	MSC_SDI_SEL_SDI4	SDI4 input is selected for SDI.
	MSC_SDI_SEL_SDI5	SDI5 input is selected for SDI.
	MSC_SDI_SEL_SDI6	SDI6 input is selected for SDI.
	MSC_SDI_SEL_SDI7	SDI7 input is selected for SDI.

2.4.2 Enums in CDD_Msc.h

2.4.2.1 Msc_StatusType

Enumeration	Msc_StatusType	
Description	MSC Frame and Channel states operation.	
Values	Value	Description
	MSC_E_OK	Success Std_Type return code.
	MSC_E_NOT_OK	failure/error Std_Type return code.

	MSC_CHANNEL_NOT_STARTED	Channel is not started.
	MSC_TX_BUSY	Command transmission in progress.
	MSC_TX_OK	Command transmission successful.
	MSC_TX_ERROR	Command transmission error.
	MSC_RX_IDLE	Waiting for data reception.
	MSC_RX_BUSY	Data reception in progress.
	MSC_RX_OK	Data reception successful.
	MSC_RX_ERROR	Data reception error.
	MSC_RX_OVERFLOW	Previous data overwritten by MSC current data reception.
	MSC_RX_TIMEOUT	Previous reception timeout by MSC current data reception.

2.4.2.2 MSC_RDRxIndexType

Enumeration	MSC_RDRxIndexType	
Description	Rx Index	
Values	Value	Description
	MSC_RDR0 = 0U	Rx register 0.
	MSC_RDR1 = 1U	Rx register 1.
	MSC_RDR2 = 2U	Rx register 2.
	MSC_RDR3 = 3U	Rx register 3.
	MSC_RDRx_COUNT = 4U	Rx register max count.

2.4.2.3 Msc_APIFunctionIdsType

Enumeration	Msc_APIFunctionIdsType	
Description	API functions service IDs.	
Values	Value	Description
	MSC_INIT_ID = 0x0U	Msc_Init() ID.
	MSC_START_CHANNEL_ID = 0x1U	Msc_StartChannel() ID.
	MSC_SET_DATA_FRAME_ID = 0x2U	Msc_SetDataFrame() ID.
	MSC_SEND_DATA_FRAME_ID = 0x3U	Msc_SendDataFrame() ID.
	MSC_STOP_CHANNEL_ID = 0x4U	Msc_StopChannel() ID.
	MSC_SEND_COMMAND_FRAME_ID = 0x5U	Msc_SendCommandFrame() ID.
	MSC_GET_TX_STATUS_ID = 0x6U	Msc_GetTxStatus() ID.
	MSC_GET_RX_STATUS_ID = 0x7U	Msc_GetRxStatus() ID.
	MSC_GET_RX_16BIT_STATUS_ID = 0x8U	Msc_GetRxStatus16BitFrame() ID.
	MSC_SWITCH_SDI_ID = 0x9U	Msc_SwitchSDIChannel() ID.
	MSC_SWITCH_EN_ID = 0xAU	Msc_SwitchEN() ID.
	MSC_GETVERSIONINFO_ID = 0xBU	Msc_GetVersionInfo() ID.
	MSC_DE_INIT_ID = 0xCU	Msc_Init() ID.

2.5 Structures

2.5.1 Msc_TransmissionSourceConfigType

Structure	Msc_TransmissionSourceConfigType
Description	Transmisson source selection.
Diagram	None
Data Fields	• Uint8 u8DataSource. Sourece Type.

2.5.2 Msc_ChannelConfigType

Structure	Msc_ChannelConfigType
Description	MSC channel configuration type structure.
Diagram	<pre> classDiagram class Msc_ChannelConfigType class Msc_TransmissionSourceConfigType Msc_ChannelConfigType ..> Msc_TransmissionSourceConfigType : tTransSelConfig </pre>
Data Fields	• uint32 u32ChannelCoreId. MSC Channel core id. • uint8 u8AllocatedPartition. MSC Channel is allocated partition or not. • uint8 u8ChannelId. MSC channel ID. • uint8 u8PassiveLength. Frame Passive length. • uint8 u8PTFNumber. The number of passive time frames that are inserted in repetition mode between two data frames. • Msc_TransmissionTriggerType eWorkMode. MSC work mode. • uint8 u8SelSRH. Select bit SRH. • uint8 u8SRHDataBitLength. Bit length of SRH. • uint8 u8SelSRL. Select bit SRL. • uint8 u8SRldataBitLength. Bit length of SRL. • uint8 u8DelayControl. Hardware Receive Interrupt Delay Control. • Msc_RxDataParityType eParity. Parity mode. • MSC_ReceiveBaudDivType eBaudDiv. Baudrate. • MSC_ReceiveFrameType eRsvFrameType. Frame type. • uint8 u8SDIPolarity. Input polarity. • MSC_SDISElectionType eSDI sel. SDI seletion. • Msc_ENxActiveType eENC. ENC seletion. • Msc_ENxActiveType eENL. ENL seletion. • Msc_ENxActiveType eENH. ENH seletion. • Msc_FclActiveType eFclCtrl. FCL Control. • uint32 u32Emergencyload. Emergency load enable. • Msc_TransmissionPolarityType ePolarity. Polarity. • uint8 u8TOIntEnable. Enable the interrupt of timeout. • uint16 u16TimeoutValue. Timeout value of timeout interrupt. • uint8 u8InjEnable0. Injection0 enable. • uint8 u8InjPosition0. Injection0 Position.

- uint8 u8InjEnable1. Injection1 enable.
- uint8 u8InjPosition1. Injection1 Position.
- Msc_TransmissionSourceConfigType tTransSelConfig..

2.5.3 Msc_ConfigType

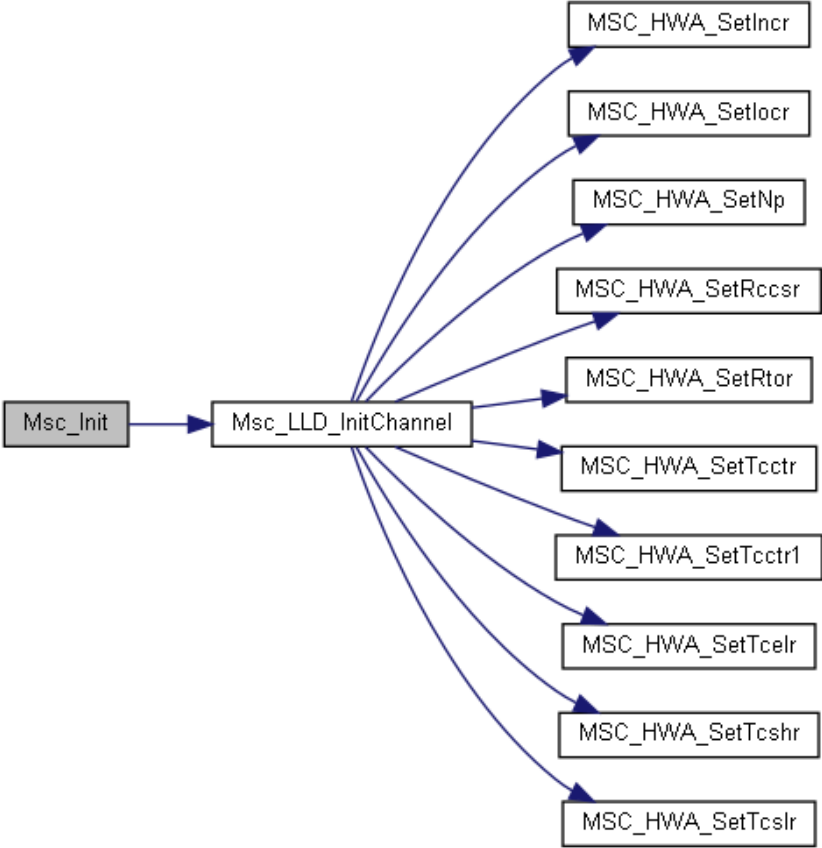
Structure	Msc_ConfigType
Description	MSC configuration type structure.
Diagram	<pre> classDiagram class Msc_TransmissionSourceConfigType class Msc_ChannelConfigType class Msc_ConfigType Msc_ChannelConfigType -- > Msc_TransmissionSourceConfigType : tTransSelConfig Msc_ConfigType -- > Msc_ChannelConfigType : Msc_Channel </pre>
Data Fields	• Uint32 PartitionCoreId. Partition ID. • Msc_ChannelConfigType *const Msc_Channel[MSC_HW_MAX_MODULES]. Channel configs.

2.6 API Functions

2.6.1 Functions in CDD_Msc.h

2.6.1.1 void Msc_Init(const Msc_ConfigType *Config);

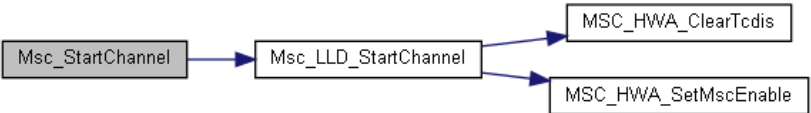
Function	void Msc_Init(const Msc_ConfigType *Config);
Description	Initializes the MSC module.

Diagram	 <pre> graph LR Msc_Init[Msc_Init] --> Msc_LLD_InitChannel[Msc_LLD_InitChannel] Msc_LLD_InitChannel --> MSC_HWA_SetIncr[MSC_HWA_SetIncr] Msc_LLD_InitChannel --> MSC_HWA_SetIocr[MSC_HWA_SetIocr] Msc_LLD_InitChannel --> MSC_HWA_SetNp[MSC_HWA_SetNp] Msc_LLD_InitChannel --> MSC_HWA_SetRccsr[MSC_HWA_SetRccsr] Msc_LLD_InitChannel --> MSC_HWA_SetRtor[MSC_HWA_SetRtor] Msc_LLD_InitChannel --> MSC_HWA_SetTcctr[MSC_HWA_SetTcctr] Msc_LLD_InitChannel --> MSC_HWA_SetTcctr1[MSC_HWA_SetTcctr1] Msc_LLD_InitChannel --> MSC_HWA_SetTcelr[MSC_HWA_SetTcelr] Msc_LLD_InitChannel --> MSC_HWA_SetTcshr[MSC_HWA_SetTcshr] Msc_LLD_InitChannel --> MSC_HWA_SetTcslr[MSC_HWA_SetTcslr] </pre>	
Parameters	Parameter	Description
	Config	Pointer to MSC driver configuration set.
Returns	void	

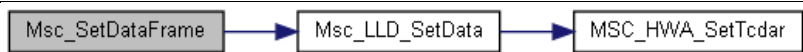
2.6.1.2 void Msc_DelInit(void);

Function	void Msc_DelInit(void);	
Description	DeInitializes the MSC module..	
Diagram	 <pre> graph LR Msc_DelInit[Msc_DelInit] --> Msc_LLD_DelInitChannel[Msc_LLD_DelInitChannel] Msc_LLD_DelInitChannel --> Msc_LLD_StopChannel[Msc_LLD_StopChannel] Msc_LLD_StopChannel --> MSC_HWA_SetMscEnable[MSC_HWA_SetMscEnable] Msc_LLD_StopChannel --> MSC_HWA_SetTcdis[MSC_HWA_SetTcdis] </pre>	
Parameters	N/A	
Returns	void	

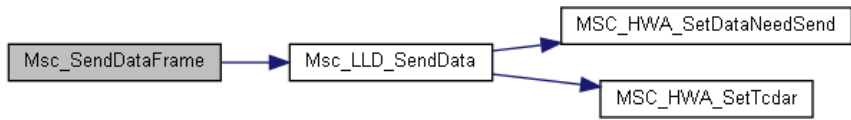
2.6.1.3 Std_ReturnType Msc_StartChannel(uint8 Channel);

Function	Std_ReturnType Msc_StartChannel(uint8 Channel);	
Description	Start to transmit serialized PWM.	
Diagram	 <pre> graph LR Msc_StartChannel[Msc_StartChannel] --> Msc_LLD_StartChannel[Msc_LLD_StartChannel] Msc_LLD_StartChannel --> MSC_HWA_ClearTcdis[MSC_HWA_ClearTcdis] Msc_LLD_StartChannel --> MSC_HWA_SetMscEnable[MSC_HWA_SetMscEnable] </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
Returns	N/A	

2.6.1.4 Std_ReturnType Msc_SetDataFrame(uint8 Channel, uint32 *Msc_DataBufferPtr);

Function	Std_ReturnType Msc_SetDataFrame(uint8 Channel, uint32 *Msc_DataBufferPtr);	
Description	Set a Data on the MSC bus.	
Diagram	 <pre> graph LR A[Msc_SetDataFrame] --> B[Msc_LLD_SetData] B --> C[MSC_HWA_SetTcdar] </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
	Msc_DataBufferPtr	Pointer to data to be transmitter in the data frame.
Returns	N/A	

2.6.1.5 Std_ReturnType Msc_SendDataFrame(uint8 Channel, uint32 *Msc_DataBufferPtr);

Function	Std_ReturnType Msc_SendDataFrame(uint8 Channel, uint32 *Msc_DataBufferPtr);	
Description	Send a Data on the MSC bus..	
Diagram	 <pre> graph LR A[Msc_SendDataFrame] --> B[Msc_LLD_SendData] B --> C[MSC_HWA_SetDataNeedSend] B --> D[MSC_HWA_SetTcdar] </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
	Msc_DataBufferPtr	Pointer to data to be transmitter in the data frame.
Returns	N/A	

2.6.1.6 Std_ReturnType Msc_StopChannel(uint8 Channel);

Function	Std_ReturnType Msc_StopChannel(uint8 Channel);	
Description	Stop transmitting serialized PWM..	
Diagram	 <pre> graph LR A[Msc_StopChannel] --> B[Msc_LLD_StopChannel] B --> C[MSC_HWA_SetMscEnable] B --> D[MSC_HWA_SetTcdis] </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
Returns	N/A	

2.6.1.7 Std_ReturnType Msc_SendCommandFrame(uint8 Channel, uint32 *Msc_CommandBufferPtr, uint8 Msc_CommandBufferBitLength);

Function	Std_ReturnType Msc_SendCommandFrame(uint8 Channel, uint32 *Msc_CommandBufferPtr, uint8 Msc_CommandBufferBitLength);	
Description	Send a command on the MSC bus.	
Diagram	 <pre> graph LR A[Msc_SendCommandFrame] --> B[Msc_LLD_SendCommand] B --> C[MSC_HWA_SetTccor] B --> D[MSC_HWA_SetTctrNbs] </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
	Msc_CommandBufferPtr	Pointer to command to be transmitter in the command frame.
	Msc_CommandBufferBitLength	length in bits of the command frame.
Returns	N/A	

Requirements	SWS_CDD_Msc_00048
--------------	-------------------

2.6.1.8 Msc_StatusType Msc_GetTxStatus(uint8 Channel);

Function	Msc_StatusType Msc_GetTxStatus(uint8 Channel);	
Description	Get the status of the command transmission on the MSC bus.	
Diagram	<pre> graph LR Msc_GetTxStatus --> Msc_LLD_GetTxStatus Msc_LLD_GetTxStatus --> MSC_HWA_GetCfb Msc_LLD_GetTxStatus --> MSC_HWA_GetCmdNeedSend </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be initialized.
Returns	Msc_ReturnType .	

2.6.1.9 Msc_StatusType Msc_GetRxStatus(uint8 Channel, uint16 **RxDataBufferPtr);

Function	Msc_StatusType Msc_GetRxStatus(uint8 Channel, uint16 **RxDataBufferPtr);	
Description	Get the status of a reception on the MSC bus.	
Diagram	<pre> graph LR Msc_GetRxStatus --> Msc_LLD_GetRxStatus Msc_LLD_GetRxStatus --> MSC_HWA_ClearRdrxVld Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxAddr Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxData Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxRerr Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxVld </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be checked.
	RxDataBufferPtr	RxDataBufferPtr pointer to pointer to a shadow buffer or memory mapped MSC Hardware receive buffer where the current SDU is stored.
Returns	Msc_ReturnType .	

2.6.1.10 Msc_StatusType Msc_GetRxStatus16BitFrame(uint8 Channel, MSC_RDRxIndexType RxIndex, uint16 **RxDataBufferPtr, uint8 *LogicAddr);

Function	Msc_StatusType Msc_GetRxStatus16BitFrame(uint8 Channel, MSC_RDRxIndexType RxIndex, uint16 **RxDataBufferPtr, uint8 *LogicAddr);	
Description	Get the status of a reception on the MSC bus..	
Diagram	<pre> graph LR Msc_GetRxStatus16BitFrame --> Msc_LLD_GetRxErrorStatus Msc_GetRxStatus16BitFrame --> Msc_LLD_GetRxStatus Msc_LLD_GetRxErrorStatus --> MSC_HWA_GetRdrxVld Msc_LLD_GetRxErrorStatus --> MSC_HWA_GetRdrxRerr Msc_LLD_GetRxStatus --> MSC_HWA_ClearRdrxVld Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxAddr Msc_LLD_GetRxStatus --> MSC_HWA_GetRdrxData </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be checked.
	RxIndex	MSC RxIndex to be checked.

	RxDataBufferPtr	RxDataBufferPtr pointer to pointer to a shadow buffer or memory mapped MSC Hardware receive buffer where the current SDU is stored.
Returns	Msc_ReturnType .	

2.6.1.11 Std_ReturnType Msc_SwitchSDIChannel(uint8 Channel, MSC_SDISelectionType SDIChannel);

Function	Std_ReturnType Msc_SwitchSDIChannel(uint8 Channel, MSC_SDISelectionType SDIChannel);	
Description	Set the SDI channel.	
Diagram	<pre> graph LR Msc_SwitchSDIChannel --> Msc_LLD_SwitchSDIChannel Msc_LLD_SwitchSDIChannel --> MSC_HWA_Getlocr Msc_LLD_SwitchSDIChannel --> MSC_HWA_GetRccsrRc Msc_LLD_SwitchSDIChannel --> MSC_HWA_Setlocr </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be checked.
	SDIChannel	Rx channel to be set.
Returns	Msc_ReturnType .	

2.6.1.12 Std_ReturnType Msc_SwitchEN(uint8 Channel, Msc_ENxType Enx, Msc_ENxActiveType ENn);

Function	Std_ReturnType Msc_SwitchEN(uint8 Channel, Msc_ENxType Enx, Msc_ENxActiveType ENn);	
Description	Set the EN channel.	
Diagram	<pre> graph LR Msc_SwitchEN --> Msc_LLD_SwitchEN Msc_LLD_SwitchEN --> MSC_HWA_GetCfb Msc_LLD_SwitchEN --> MSC_HWA_GetCmdNeedSend Msc_LLD_SwitchEN --> MSC_HWA_GetDataNeedSend Msc_LLD_SwitchEN --> MSC_HWA_GetDfb Msc_LLD_SwitchEN --> MSC_HWA_Getlocr Msc_LLD_SwitchEN --> MSC_HWA_Setlocr </pre>	
Parameters	Parameter	Description
	Channel	MSC channel to be checked.
	ENx	ENC ENH ENL channel to be set.
	ENn	EN0 EN1 EN2 EN3 to be set.
Returns	Msc_ReturnType .	

2.6.1.13 void Msc_GetVersionInfo(Std_VersionInfoType *versioninfo);

Function	void Msc_GetVersionInfo(Std_VersionInfoType *versioninfo);	
Description	Returns the version information of this module.	
Diagram	None	
Parameters	Parameter	Description
	versioninfo	Pointer for storing the version information of this module.
Returns	void.	

2.7 Hardware Functions

2.7.1 Functions in CDD_Msc_LLD.h

2.7.1.1 void Msc_LLD_InitChannel(const Msc_ChannelConfigType *pChannelConfig)

Function	void Msc_LLD_InitChannel(const Msc_ChannelConfigType *pChannelConfig)	
Description	Initialize a MSC channel.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
Returns	Std_ReturnType.	

2.7.1.2 void Msc_LLD_DeInitChannel(uint8 u8Channel)

Function	void Msc_LLD_DeInitChannel(uint8 u8Channel)	
Description	DeInitialize a MSC channel.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be De-initialized.
Returns	void.	

2.7.1.3 void Msc_LLD_SetData(uint8 u8Channel, const uint32 *DataBufferPtr)

Function	void Msc_LLD_SetData(uint8 u8Channel, const uint32 *DataBufferPtr)	
Description	Set a data on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
	DataBufferPtr	Pointer to data to be transmitter in the data frame.
Returns	void.	

2.7.1.4 void Msc_LLD_SendData(uint8 u8Channel, const uint32 *DataBufferPtr)

Function	void Msc_LLD_SendData(uint8 u8Channel, const uint32 *DataBufferPtr)	
Description	Send a data on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
	DataBufferPtr	Pointer to data to be transmitter in the data frame.
Returns	void.	

2.7.1.5 Std_ReturnType Msc_LLD_StartChannel(uint8 u8Channel)

Function	Std_ReturnType Msc_LLD_StartChannel(uint8 u8Channel)	
Description	Start to transmit serialized PWM.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
Returns	Std_ReturnType.	

2.7.1.6 Std_ReturnType Msc_LLD_StopChannel(uint8 u8Channel)

Function	Std_ReturnType Msc_LLD_StopChannel(uint8 u8Channel)	
Description	Stop transmitting serialized PWM.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
Returns	void.	

2.7.1.7 void Msc_LLD_SendCommand(uint8 u8Channel, const uint32 *CommandBufferPtr, uint8 CommandBufferBitLength)

Function	void Msc_LLD_SendCommand(uint8 u8Channel, const uint32 *CommandBufferPtr, uint8 CommandBufferBitLength)	
Description	Send a command on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
	CommandBufferPtr	Pointer to command to be transmitter in the command frame.
	CommandBufferBitLength	length in bits of the command frame.
Returns	void.	

2.7.1.8 Msc_StatusType Msc_LLD_GetTxStatus(uint8 u8Channel)

Function	Msc_StatusType Msc_LLD_GetTxStatus(uint8 u8Channel)	
Description	Get the status of the command transmission on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
Returns	Msc_StatusType .	

2.7.1.9 Msc_StatusType Msc_LLD_GetRxStatus(uint8 u8Channel, MSC_RDRxIndexType eRxindex, uint16 *pRxDataBufferPtr, uint8 *pLogicAddr)

Function	Msc_StatusType Msc_LLD_GetRxStatus(uint8 u8Channel, MSC_RDRxIndexType eRxindex, uint16 *pRxDataBufferPtr, uint8 *pLogicAddr)	
Description	Get the status of a reception on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
	pRxDataBufferPtr	Pointer to data to be received.
Returns	Msc_StatusType.	

2.7.1.10 Msc_StatusType Msc_LLD_GetRxErrorStatus(uint8 u8Channel, MSC_RDRxIndexType eRxindex)

Function	Msc_StatusType Msc_LLD_GetRxErrorStatus(uint8 u8Channel, MSC_RDRxIndexType eRxindex)	
Description	Get the status of a reception on the MSC bus.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be initialized.
Returns	Msc_StatusType.	

2.7.1.11 Std_ReturnType Msc_LLD_SwitchSDIChannel(uint8 u8Channel, MSC_SDISelectionType eSDIChannel)

Function	Std_ReturnType Msc_LLD_SwitchSDIChannel(uint8 u8Channel, MSC_SDISelectionType eSDIChannel)	
Description	Change the SDI.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be changed.
	eSDIChannel	MSC SDI channel to be selected.
Returns	Std_ReturnType.	

2.7.1.12 Std_ReturnType Msc_LLD_SwitchEN(uint8 u8Channel, Msc_ENxType eEnx, Msc_ENxActiveType eENn)

Function	Std_ReturnType Msc_LLD_SwitchEN(uint8 u8Channel, Msc_ENxType eEnx, Msc_ENxActiveType eENn)	
Description	Change the EN.	
Parameters	Parameter	Description
	u8Channel	MSC channel to be changed.
	eENx	MSC EN type to be selected.
	eENn	MSC EN channel to be selected.
Returns	Std_ReturnType.	

2.7.1.13 static void Msc_LLD_RxInterruptHandler(uint8 u8Channel)

Function	static void Msc_LLD_RxInterruptHandler(uint8 u8Channel)	
Description	Interrupt handler for RX on LINFlex.	
Parameters	Parameter	Description
	BaseAddress	of the logical MSC channel to be addressed.

2.8 Peripheral Functions

2.8.1 Functions in CDD_Msc_Hw.h

2.8.1.1 LOCAL_INLINE void MSC_HWA_SetTcctr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcctr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Get the msc TCCTR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCTR register value
Returns	N/A	

2.8.1.2 LOCAL_INLINE void MSC_HWA_SetTcctrNbs(MSC_Type *const pMsc, uint8 u8Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcctrNbs(MSC_Type *const pMsc, uint8 u8Value)	
Description	Get the msc TCCTR register NBS.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCTR register value

Returns	N/A
----------------	-----

2.8.1.3 LOCAL_INLINE uint8 MSC_HWA_GetDataNeedSend(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetDataNeedSend(const MSC_Type *const pMsc)	
Description	Get the msc TCCTR register DTS bit.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	DTS value	

2.8.1.4 LOCAL_INLINE void MSC_HWA_SetDataNeedSend(MSC_Type *const pMsc)

Function	LOCAL_INLINE void MSC_HWA_SetDataNeedSend(MSC_Type *const pMsc)	
Description	Set the msc TCCTR register DTS bit.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	N/A	

2.8.1.5 LOCAL_INLINE uint8 MSC_HWA_GetCmdNeedSend(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetCmdNeedSend(const MSC_Type *const pMsc)	
Description	Get the msc TCCTR register CTS bit.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	CTS value	

2.8.1.6 LOCAL_INLINE uint8 MSC_HWA_GetCfb(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetCfb(const MSC_Type *const pMsc)	
Description	Get the CFB value of TCSTR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	CFB value	

2.8.1.7 LOCAL_INLINE uint8 MSC_HWA_GetDfb(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetDfb(const MSC_Type *const pMsc)	
Description	Get the DFB value of TCSTR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	DFB value	

2.8.1.8 LOCAL_INLINE void MSC_HWA_SetNp(MSC_Type *const pMsc, uint8 u8Np)

Function	LOCAL_INLINE void MSC_HWA_SetNp(MSC_Type *const pMsc, uint8 u8Np)	
Description	Set the NP value of TCSTR register.	
Parameters	Parameter	Description

	pMsc	MSCInstance
Returns	N/A	

2.8.1.9 LOCAL_INLINE void MSC_HWA_SetTcdar(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcdar(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the msc TCDAR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCDAR value
Returns	N/A	

2.8.1.10 LOCAL_INLINE void MSC_HWA_SetTccor(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTccor(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the TCCOR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCOR value
Returns	N/A	

2.8.1.11 LOCAL_INLINE void MSC_HWA_SetTcslr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcslr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the TCCLR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCLR value
Returns	N/A	

2.8.1.12 LOCAL_INLINE void MSC_HWA_SetTcshr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcshr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the TCCLR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCLR value
Returns	N/A	

2.8.1.13 LOCAL_INLINE void MSC_HWA_SetTcelr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcelr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the TCELR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCELR value

Returns	N/A
----------------	-----

2.8.1.14 LOCAL_INLINE void MSC_HWA_Setloclr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_Setloclr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the IOCR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	IOCR value
Returns	N/A	

2.8.1.15 LOCAL_INLINE uint32 MSC_HWA_Getloclr(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint32 MSC_HWA_Getloclr(const MSC_Type *const pMsc)	
Description	Get the msc IOCR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	INSR register value	

2.8.1.16 LOCAL_INLINE void MSC_HWA_SetTcdis(MSC_Type *const pMsc)

Function	LOCAL_INLINE void MSC_HWA_SetTcdis(MSC_Type *const pMsc)	
Description	Set the msc TCDIS.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	N/A	

2.8.1.17 LOCAL_INLINE void MSC_HWA_ClearTcdis(MSC_Type *const pMsc)

Function	LOCAL_INLINE void MSC_HWA_ClearTcdis(MSC_Type *const pMsc)	
Description	Clear the msc TCDIS.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	N/A	

2.8.1.18 LOCAL_INLINE void MSC_HWA_ClearCrfi(MSC_Type *const pMsc)

Function	LOCAL_INLINE void MSC_HWA_ClearCrfi(MSC_Type *const pMsc)	
Description	Clear the msc CRFI.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	N/A	

2.8.1.19 LOCAL_INLINE uint8 MSC_HWA_GetRfi(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRfi(const MSC_Type *const pMsc)	
Description	Get the msc INSR register RFI.	

Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	INSR register value	

2.8.1.20 LOCAL_INLINE void MSC_HWA_SetIncr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetIncr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the INCR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	INCR value
Returns	N/A	

2.8.1.21 LOCAL_INLINE uint8 MSC_HWA_GetRccsrRc(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRccsrRc(const MSC_Type *const pMsc)	
Description	Get the msc RCCSR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	RCCSR register value	

2.8.1.22 LOCAL_INLINE void MSC_HWA_SetRccsr(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetRccsr(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the RCCSR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	RCCSR value
Returns	N/A	

2.8.1.23 LOCAL_INLINE uint32 MSC_HWA_GetRdrxRerr(const volatile uint32 *pRdrx)

Function	LOCAL_INLINE uint32 MSC_HWA_GetRdrxRerr(const volatile uint32 *pRdrx)	
Description	Get the RERR of RDRx register.	
Parameters	Parameter	Description
	pRdrx	MSC rx data register
Returns	RERR value	

2.8.1.24 LOCAL_INLINE uint8 MSC_HWA_GetRdrxData(const volatile uint32 *pRdrx)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRdrxData(const volatile uint32 *pRdrx)	
Description	Get the DATA of RDR1 register.	
Parameters	Parameter	Description
	pRdrx	MSC rx data register
Returns	RDATA value	

2.8.1.25 LOCAL_INLINE uint8 MSC_HWA_GetRdrxAddr(const volatile uint32 *pRdrx)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRdrxAddr(const volatile uint32 *pRdrx)	
Description	Get the LAF of RDRx register.	
Parameters	Parameter	Description
	pRdrx	MSC rx data register
Returns	LAF value	

2.8.1.26 LOCAL_INLINE uint8 MSC_HWA_GetRdrxVld(const volatile uint32 *pRdrx)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRdrxVld(const volatile uint32 *pRdrx)	
Description	Get the VLD of RDRx register.	
Parameters	Parameter	Description
	pRdrx	MSC rx data register
Returns	LAF value	

2.8.1.27 LOCAL_INLINE void MSC_HWA_ClearRdrxVld(volatile uint32 *pRdrx)

Function	LOCAL_INLINE void MSC_HWA_ClearRdrxVld(volatile uint32 *pRdrx)	
Description	Clear the VLD of register.	
Parameters	Parameter	Description
	pRdrx	MSC rx data register
Returns	N/A	

2.8.1.28 LOCAL_INLINE uint8 MSC_HWA_GetRtorToie(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRtorToie(const MSC_Type *const pMsc)	
Description	Get the timeout interrupt enable.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	reset status	

2.8.1.29 LOCAL_INLINE uint8 MSC_HWA_GetRtorTof(const MSC_Type *const pMsc)

Function	LOCAL_INLINE uint8 MSC_HWA_GetRtorTof(const MSC_Type *const pMsc)	
Description	Get the timeout interrupt flag.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	reset status	

2.8.1.30 LOCAL_INLINE void MSC_HWA_ClearRtorTof(MSC_Type *const pMsc)

Function	LOCAL_INLINE void MSC_HWA_ClearRtorTof(MSC_Type *const pMsc)	
Description	Clear the timeout interrupt flag.	
Parameters	Parameter	Description
	pMsc	MSCInstance
Returns	N/A	

Requirements	
---------------------	--

2.8.1.31 LOCAL_INLINE void MSC_HWA_SetRtor(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetRtor(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the msc RTOR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	RTOR value
Returns	N/A	

2.8.1.32 LOCAL_INLINE void MSC_HWA_SetTcctr1(MSC_Type *const pMsc, uint32 u32Value)

Function	LOCAL_INLINE void MSC_HWA_SetTcctr1(MSC_Type *const pMsc, uint32 u32Value)	
Description	Set the msc TCCTR1 register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	u32Value	TCCTR1 value
Returns	N/A	

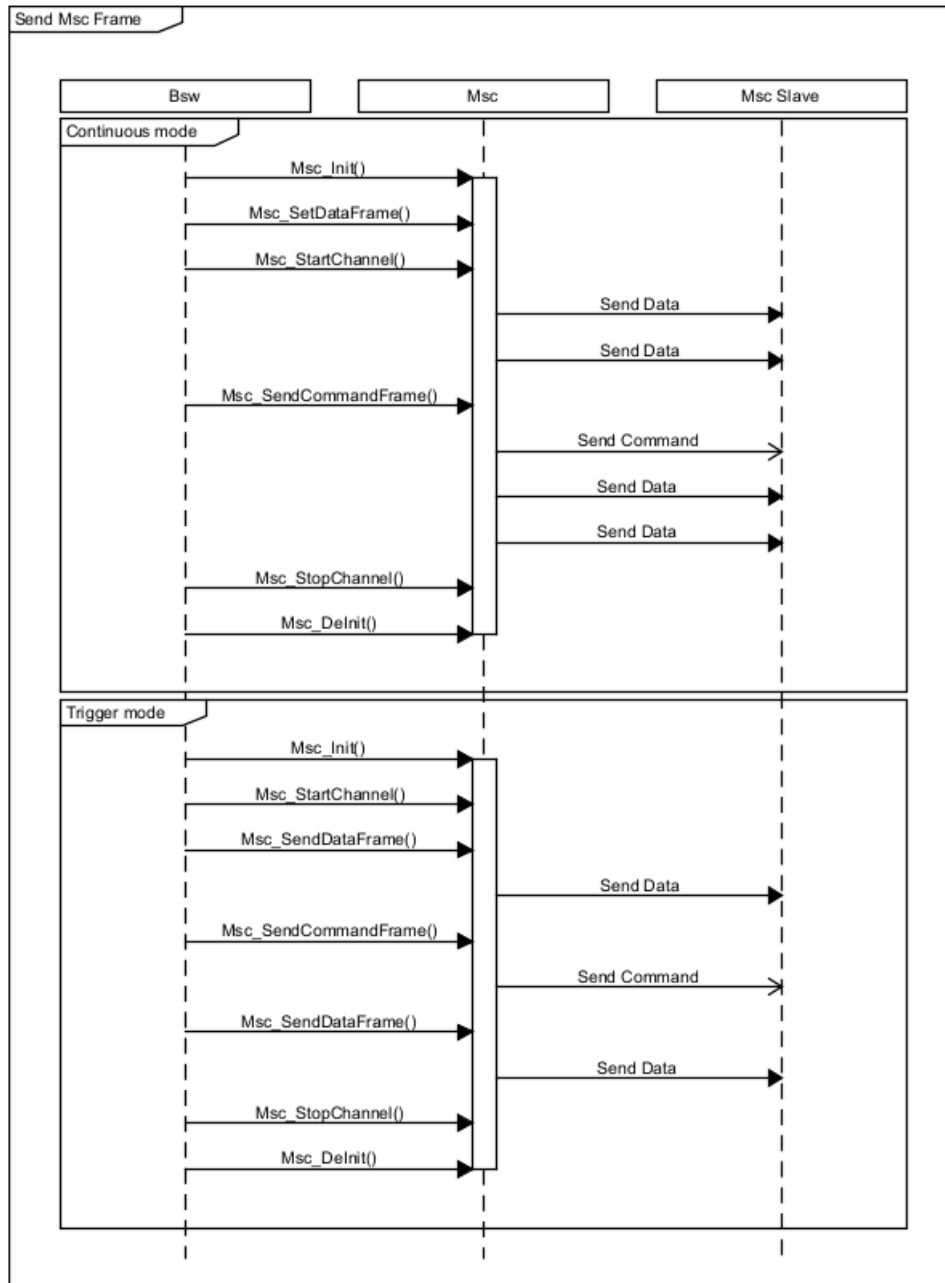
2.8.1.33 LOCAL_INLINE void MSC_HWA_SetMscEnable(MSC_Type *const pMsc, uint8 u8Enable)

Function	LOCAL_INLINE void MSC_HWA_SetMscEnable(MSC_Type *const pMsc, uint8 u8Enable)	
Description	Set the EN msc GCR register.	
Parameters	Parameter	Description
	pMsc	MSCInstance
	bEnable	EN value
Returns	N/A	

2.9 API Sequence Diagram

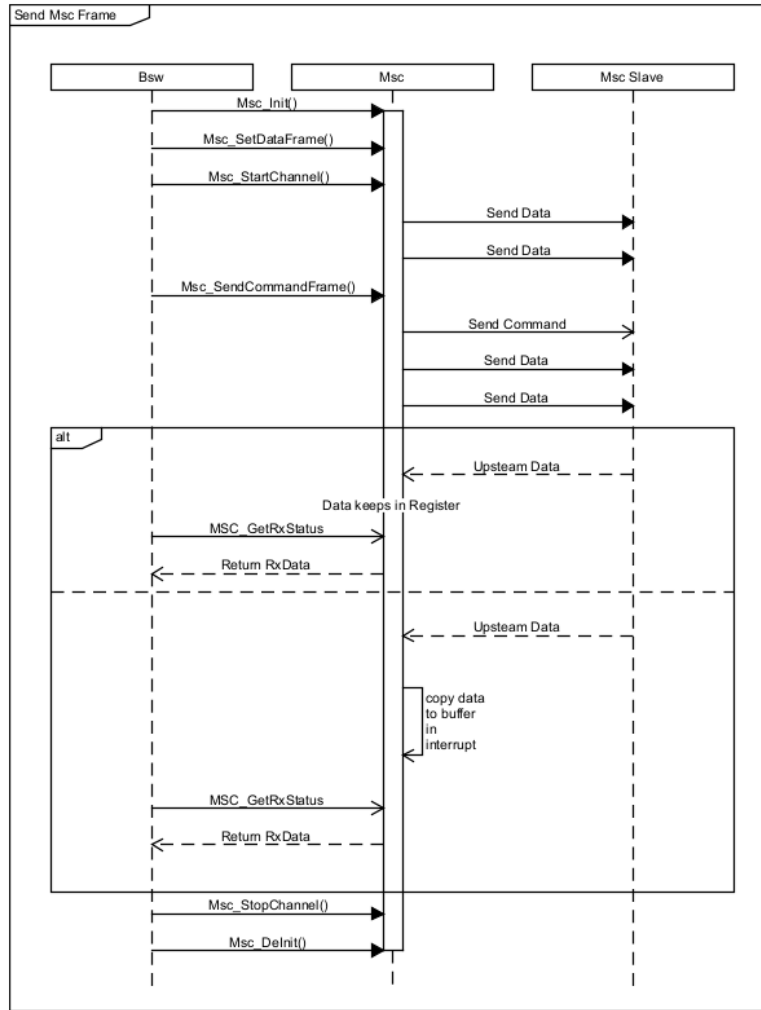
2.9.1 Frame Transmission

In Continuous Mode, the Msc_Init function is first called to initialize the Msc peripheral, and then Msc_SetDataFrame is called to set the Data register. After calling Msc_StartChannel, the Msc starts periodic transmission of Data frames. The upper layer sends the Command frame after calling Msc_SendCommandFrame. If it is Trigger Mode, then the Msc will not actively send Data frames until the upper layer calls Msc_SendDataFrame.



2.9.2 Frame Reception

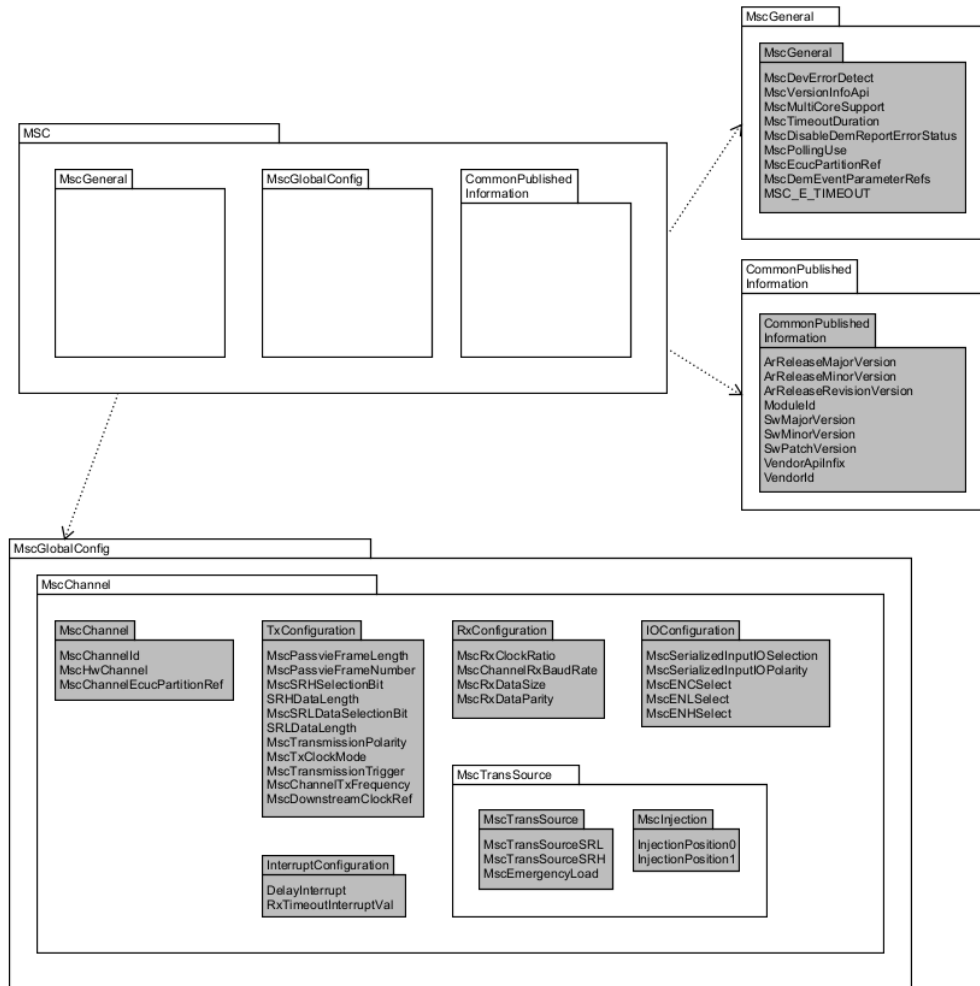
The following use case shows the reception of a MSC frame. When the Msc receives upstream data frame, it can choose whether to use interrupts to process the data. If polling mode is enabled, then the Data needs to be read from the Rx Data register of the Msc by calling `Msc_GetRxStatus` from the upper application. If the interrupt is enabled, the Data Frame is copied into the buffer during the interrupt, and the upper layer obtains the Data frame from the buffer by calling `Msc_GetRxStatus`.



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The container inclusion relation is shown below:



3.2 Containers and Variables

3.2.1 MscGeneral

Container	MscGeneral	
Description	MscGeneral This container contains the parameters related to each MSC Driver Unit.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.1.1 Msc Development Error Detection

Variable	MscDevErrorDetect
----------	-------------------

Description	MscDevErrorDetect Switches the Development Error Detection and Notification ON or OFF.	
Screenshot	Msc Development Error Detection 	
Properties	Property	Value
	Type	BOOLEAN
	SynmbolicName	false
	Default	false
	Origin	Flagchip

3.2.1.2 Provide Msc VersionInfo Api

Variable	MscVersionInfoApi	
Description	MscVersionInfoApi Switches the Msc_GetVersionInfo function ON or OFF.	
Screenshot	Provide Msc VersionInfo Api 	
Properties	Property	Value
	Type	BOOLEAN
	SynmbolicName	false
	Default	false
	Origin	Flagchip

3.2.1.3 Msc MultiCore Support

Variable	MscMultiCoreSupport	
Description	MscMultiCoreSupport Switches the Multi core function ON or OFF.	
Screenshot	Msc MultiCore Support 	
Properties	Property	Value
	Type	BOOLEAN
	SynmbolicName	false
	Default	false
	Origin	Flagchip

3.2.1.4 Msc Timeout Duration

Variable	MscTimeoutDuration	
Description	MscTimeoutDuration Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops	
Screenshot	Msc Timeout Duration (0 -> 1000000) 	
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	10000
	Origin	Flagchip

3.2.1.5 Msc Disable Production Error Reporting

Variable	MscDisableDemReportErrorStatus	
-----------------	--------------------------------	--

Description	MscDisableDemReportErrorStatus Switches the Diagnostic Error Reporting and Notification OFF.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SynmbolicName	false
	Default	false
	Origin	Flagchip

3.2.1.6 Msc Use Polling

Variable	MscPollingUse	
Description	MscPollingUse Use polling instead of interrupt to receive data.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SynmbolicName	false
	Default	true
	Origin	Flagchip

3.2.1.7 MscEcucPartitionRef

Variable	MscEcucPartitionRef	
Description	Maps the Msc driver to zero or multiple ECUC partitions to make the modules API available in this partition. The Msc driver will operate as an independent instance in each of the partitions.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

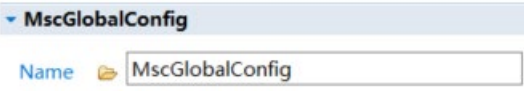
3.2.1.8 MscDemEventParameterRefs

Container	MscDemEventParameterRefs	
Description	Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

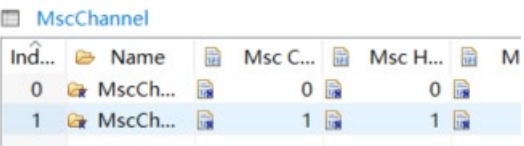
3.2.1.8.1 Msc Timeout Dem Error

Variable	MSC_E_TIMEOUT	
Description	Reference to the DemEventParameter which shall be issued when the error "Timeout caused by hardware error" has occurred.	
Screenshot		
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Origin	Flagchip

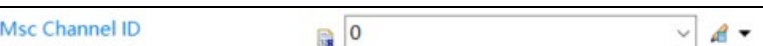
3.2.2 MscGlobalConfig

Container	MscGlobalConfig	
Description	This container contains the global configuration parameter of the Msc driver.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1 MscChannel

Container	MscChannel	
Description	This container contains the configuration (parameters) of the MSC Controller(s). Note:"User should use unique names for naming the MSC channels across different MscGlobalConfig Sets."	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.1 Msc Channel ID

Variable	MscChannelId	
Description	Identifies the MSC logical channel.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	true
	Origin	Flagchip
	Range	0-1

3.2.2.1.2 Msc Hw Channel Index

Variable	MscHwChannel	
Description	Identifies the MSC Hw channel.	
Screenshot		

Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Origin	Flagchip
	Range	0-1

3.2.2.1.3 MscChannelEcucPartitionRef

Variable	MscChannelEcucPartitionRef	
Description	Maps one single Msc channel to zero or one ECUC partitions. The ECUC partition referenced is a subset of the ECUC partitions where the Msc driver is mapped to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.4 TxConfiguration

Container	TxConfiguration	
Description		
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.4.1 Msc Passive Frame Length(bit)

Variable	MscPassvieFrameLength	
Description	MscPassvieFrameLength Specifies the passive frame length."	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	2
	Origin	Flagchip

3.2.2.1.4.2 Msc Passive Frame Number

Variable	MscPassvieFrameNumber	
Description	MscPassvieFrameLength Specifies the passive frame Number."	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	1

	Origin	Flagchip
--	--------	----------

3.2.2.1.4.3 Msc SRH Data Selection Bit

Variable	MscSRHSelectionBit	
Description	Select if the SRH DataFrame contain a data selection bit. MSC_DATA_SELECTION_BIT_ENABLE: DataFrame contain a data selection bit. MSC_DATA_SELECTION_BIT_DISABLE: DataFrame does not contain a data selection bit.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_DATA_SELECTION_BIT_DISABLE
	Origin	Flagchip
	Range	[MSC_DATA_SELECTION_BIT_ENABLE, MSC_DATA_SELECTION_BIT_DISABLE]

3.2.2.1.4.4 Msc SRH data length

Variable	SRHDataLength	
Description	SRHDataLength Specifies the SRH data length."	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	16
	Origin	Flagchip

3.2.2.1.4.5 Msc SRL Data Selection Bit

Variable	MscSRLDataSelectionBit	
Description	Select if the SRL DataFrame contain a data selection bit. MSC_DATA_SELECTION_BIT_ENABLE: DataFrame contain a data selection bit. MSC_DATA_SELECTION_BIT_DISABLE: DataFrame does not contain a data selection bit.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_DATA_SELECTION_BIT_ENABLE
	Origin	Flagchip
	Range	[MSC_DATA_SELECTION_BIT_ENABLE, MSC_DATA_SELECTION_BIT_DISABLE]

3.2.2.1.4.6 Msc SRL data length

Variable	SRLDataLength	
Description	SRHDataLength Specifies the SRL data length."	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	16
	Origin	Flagchip

3.2.2.1.4.7 Msc Transmission Polarity

Variable	MscTransmissionPolarity	
Description	Select the TX output polarity. MSC_TX_NORMAL_POLARITY: Clock ChipSelect and Data are active high. MSC_TX_INVERTED_POLARITY_FSC: ChipSelect and Data are active high, Clock is active low. MSC_TX_INVERTED_POLARITY_EN: Clock and Data are active high, ChipSelect is active low. MSC_TX_INVERTED_POLARITY_SO: ChipSelect and Clock are active high, Data is active low. MSC_TX_INVERTED_POLARITY_FSC_EN: Data is active high, Clock and ChipSelect are active low. MSC_TX_INVERTED_POLARITY_FSC_SO: ChipSelect is active high, Clock and Data are active low. MSC_TX_INVERTED_POLARITY_EN_SO: Clock is active high, ChipSelect and Data are active low. MSC_TX_INVERTED_POLARITY_FSC_EN_SO: Clock ChipSelect and Data are active low.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	MSC_TX_NORMAL_POLARITY
	Origin	Flagchip
	Range	[MSC_TX_NORMAL_POLARITY, MSC_TX_INVERTED_POLARITY_FSC, MSC_TX_INVERTED_POLARITY_EN, MSC_TX_INVERTED_POLARITY_FSC_EN]

3.2.2.1.4.8 Msc Tx Mode

Variable	MscTxClockMode	
Description	Select TX clock mode for the RX. MSC_TX_SYNCHRONOUS: Tx clock is transmitted always. MSC_TX_ASYNCHRONOUS: Tx clock is transmitted only on Tx.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	MSC_TX_SYNCHRONOUS
	Origin	Flagchip
	Range	[MSC_TX_SYNCHRONOUS, MSC_TX_ASYNCHRONOUS]

3.2.2.1.4.9 Msc Transmission Trigger

Variable	MscTransmissionTrigger	
Description	Defines the trigger for the Data transmission. MSC_TX_TRIGGER_SINGLE: Data transmission is triggered by SW. MSC_TX_TRIGGER_CONTINUOUS: Data is transmitted continuously based on bus capacity.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	MSC_TX_TRIGGER_SINGLE
	Origin	Flagchip
	Range	[MSC_TX_TRIGGER_SINGLE, MSC_TX_TRIGGER_CONTINUOUS]

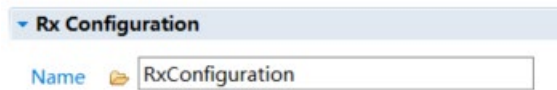
3.2.2.1.4.10 Msc Channel Tx Frequency (Hz)

Variable	MscChannelTxFrequency	
Description	MscChannelTxFrequency Specifies the clock frequency of the Msc TX channel in Hz. the FCL clock is obtained by bisecting the clock frequency."	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	Flagchip

3.2.2.1.4.11 MscDownstreamClockRef

Variable	MscDownstreamClockRef	
Description	Reference to the MSC clock source configuration for Downstream frame generation, which is set in the MCU driver configuration.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip

3.2.2.1.5 RxConfiguration

Container	RxConfiguration	
Description	Rx configuration	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.5.1 Msc Rx Clock Ratio

Variable	MscRxClockRatio
-----------------	-----------------

Description	MscRxClockRatio Rx clock ratio is derived from the synchronous Tx baud rate, dividing it by 2 exp n.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	128
	Origin	Flagchip

3.2.2.1.5.2 Msc Channel Rx baudrate

Variable	MscChannelRxBaudRate	
Description	MscChannelRxBaudRate	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Origin	Flagchip

3.2.2.1.5.3 Msc Rx Data Size

Variable	MscRxDataSize	
Description	RX data size. 8 or 12 bits.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_RX_12_BIT_LENGTH
	Origin	Flagchip
	Range	[MSC_RX_12_BIT_LENGTH, MSC_RX_16_BIT_LENGTH]

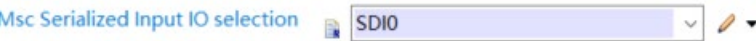
3.2.2.1.5.4 Msc Rx Data Parity

Variable	MscRxDataParity	
Description	RX data parity. Even or Odd.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_RX_PARITY_EVEN
	Origin	Flagchip
	Range	[MSC_RX_PARITY_EVEN, MSC_RX_PARITY_ODD]

3.2.2.1.6 IOConfiguration

Container	IOConfiguration	
Description	IO Configuration.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.6.1 Msc Serialized Input IO selection

Variable	MscSerializedInputIOSelection	
Description	MscSerializedInputSelection Define.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	SDI0
	Origin	Flagchip
	Range	[SDI0, SDI1, SDI2, SDI3, SDI4, SDI5, SDI6, SDI7]

3.2.2.1.6.2 Msc Serialized Input IO Polarity

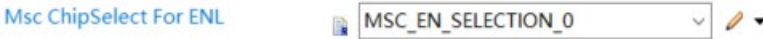
Variable	MscSerializedInputIOPolarity	
Description	MscSerializedInputPolarity Define the input polarity.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	SDI_POLARITY_IDENTICAL
	Origin	Flagchip
	Range	[SDI_POLARITY_IDENTICAL, SDI_POLARITY_INVERTED]

3.2.2.1.6.3 Msc ChipSelect For ENC

Variable	MscENCSelect	
Description	Select the EN0 select for the downstream frame.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_EN_SELECTION_0
	Origin	Flagchip

	Range	[MSC_EN_SELECTION_0, MSC_EN_SELECTION_1, MSC_EN_SELECTION_2, MSC_EN_SELECTION_3]
--	-------	--

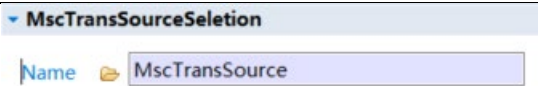
3.2.2.1.6.4 Msc ChipSelect For ENL

Variable	MscENLSelect	
Description	Select the ENL select for the downstream frame.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_EN_SELECTION_0
	Origin	Flagchip
	Range	[MSC_EN_SELECTION_0, MSC_EN_SELECTION_1, MSC_EN_SELECTION_2, MSC_EN_SELECTION_3]

3.2.2.1.6.5 Msc ChipSelect For ENH

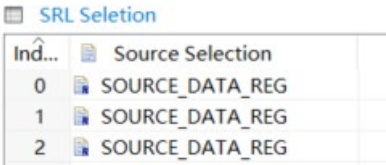
Variable	MscENHSelect	
Description	Select the ENH select for the downstream frame.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	MSC_EN_SELECTION_0
	Origin	Flagchip
	Range	[MSC_EN_SELECTION_0, MSC_EN_SELECTION_1, MSC_EN_SELECTION_2, MSC_EN_SELECTION_3]

3.2.2.1.7 MscTransSource

Container	MscTransSource	
Description	Transmission source selection	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.7.1 Source Selection

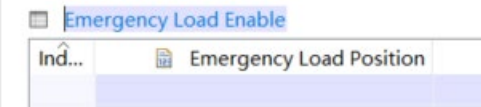
Variable	MscTransSourceSRL
Description	SRL Source Selection

Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynbolicName	false
	Default	SOURCE_DATA_REG
	Origin	Flagchip
	Range	[SOURCE_DATA_REG, SOURCE_DATA_ALTIN, SOURCE_DATA_ALTIN_INV]

3.2.2.1.7.2 Source Selection

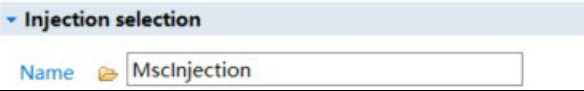
Variable	MscTransSourceSRH	
Description	SRH Source Selection	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynbolicName	false
	Default	SOURCE_DATA_REG
	Origin	Flagchip
	Range	[SOURCE_DATA_REG, SOURCE_DATA_ALTIN, SOURCE_DATA_ALTIN_INV]

3.2.2.1.7.3 Emergency Load Position

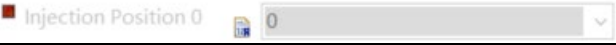
Variable	MscEmergencyLoad	
Description	Emergency Load Position.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynbolicName	false
	Default	0
	Origin	Flagchip
	Range	0-31

3.2.2.1.7.4 MscInjection

Container	MscInjection	
Description	Injection selection.	

Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

Injection Position 0

Variable	InjectionPosition0	
Description	Injection position 0.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip
	Range	0-31

Injection Position 1

Variable	InjectionPosition1	
Description	Injection position 1.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip
	Range	0-31

3.2.2.1.8 InterruptConfiguration

Container	InterruptConfiguration	
Description	InterruptConfiguration	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.8.1 Msc Interrupt Delay

Variable	DelayInterrupt	
Description	DelayInterrupt Delay the interrupt for 1 bit.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN

	SynmbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.8.2 Msc Rx Timeout Value

Variable	RxTimeoutInterruptVal	
Description	Rx Timeout Interrupt Value If this value is set, rx state will indicate timeout. The timeout unit is in rx bits.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	100
	Origin	Flagchip

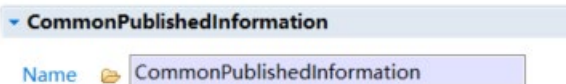
3.2.2.1.8.3 RxTimeoutNotification

Variable	RxTimeoutNotification	
Description	Rx Timeout Interrupt Notification Callback.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SynmbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

3.2.2.1.8.4 RxReceiveNotification

Variable	RxReceiveNotification	
Description	Rx Interrupt Notification Callback.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SynmbolicName	False
	Default	NULL_PRT
	Origin	Flagchip

3.2.3 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		

Properties	Property	Value
	Type	IDENTIFIABLE

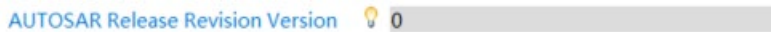
3.2.3.1 AUTOSAR Major Version

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	4
	Origin	Flagchip

3.2.3.2 AUTOSAR Minor Version

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	6
	Origin	Flagchip

3.2.3.3 AUTOSAR Release Revision Version

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.4 Numeric Module ID

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false

	Default	253
	Origin	Flagchip

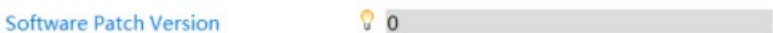
3.2.3.5 Software Major Version

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.6 Software Minor Version

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	3
	Origin	Flagchip

3.2.3.7 Software Patch Version

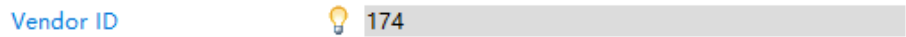
Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.8 Vendor Api Infix

Variable	VendorApiInfix	
Description	In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:	

	<ModuleName>_>VendorId>_<VendorApiInfix><Api name from SWS>. E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write. This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.	
Screenshot	 Vendor Api Infix 	
Properties	Property	Value
	Type	STRING_LABEL
	SynmbolicName	false
	Origin	Flagchip

3.2.3.9 Vendor ID

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot	 Vendor ID  174	
Properties	Property	Value
	Type	INTEGER_LABEL
	SynmbolicName	false
	Default	174
	Origin	Flagchip

Chapter 4 Configuration Guides

4.1 MSC Usage Common Steps

Basically, the MSC module can be configured by following the below 3 steps:

- 1) Configure MCU to enable MSC clock.
- 2) Enable MSC pins in Port configuration.
- 3) Configure PWM if the transmission source is FTU channels.
- 4) Configure MSC general configurations.
- 5) Generate configuration files.

4.2 MSC Channel Demo

- 1) Configure MCU to enable MSC clock

McuClockSettingConfig

Name:

General FOSC SOSC SIRC SIRC32K FIRC PLL0 PLL1 System Clock **PCC** Clock Reference Point CSC0 SCM

McuPeripheralClockConfig

Ind...	Name	Mcu Peripheral Name	Enable...	Domai...	Periph...	Periph...	Periph...
33	McuPer...	TRGSEL1	☒	ALL_CP...	OFF	1	0.0
34	McuPer...	TRGSEL2	☒	ALL_CP...	OFF	1	0.0
35	McuPer...	TRGSEL3	☒	ALL_CP...	OFF	1	0.0
36	McuPer...	FCSPI3	☒	ALL_CP...	FOSCDI...	1	1.2E7
37	McuPer...	WKU0	☒	ALL_CP...	OFF	1	0.0
38	McuPer...	FCSPI0	☒	ALL_CP...	OFF	1	0.0
39	McuPer...	FCUART7	☒	ALL_CP...	FOSCDI...	1	1.2E7
40	McuPer...	FCSPI2	☒	ALL_CP...	FOSCDI...	1	1.2E7
41	McuPer...	DMA0	☒	ALL_CP...	OFF	1	0.0
42	McuPer...	ROMC	☒	ALL_CP...	OFF	1	0.0
43	McuPer...	RTC	☒	ALL_CP...	OFF	1	0.0
44	McuPer...	FTU7	☒	ALL_CP...	OFF	1	0.0
45	McuPer...	FCSPI6	☒	ALL_CP...	FOSCDI...	1	1.2E7
46	McuPer...	FCSPI7	☒	ALL_CP...	FOSCDI...	1	1.2E7
47	McuPer...	FCSPI5	☒	ALL_CP...	FOSCDI...	1	1.2E7
48	McuPer...	FLEXCAN8	☒	ALL_CP...	FOSCDI...	1	2.4E7
49	McuPer...	MSC0	☒	ALL_CP...	FOSCDI...	4	300000...
50	McuPer...	MSC1	☒	ALL_CP...	FOSCDI...	4	300000...

- 2) Enable MSC pins in Port configuration. For example, to enable FCLP.

General **PortPin** DigitalFilterChannel

PortPin

Ind...	Name	PortPi...	PortPi...	PortPi...	PortPi...	PortPin Mode	PortPi...	PortPi...	PortPi...	PortPi...	PortPi...	PortPi...
0	PortPin_0	0	0	PTA0	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
1	PortPin_1	1	1	PTA1	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
2	PortPin_2	2	2	PTA2	☒	MSC1_FCLP	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
3	PortPin_3	3	3	PTA3	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
4	PortPin_4	4	4	PTA4	☒	JTAG_TMS_S...	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
5	PortPin_5	6	6	PTA6	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
6	PortPin_6	7	7	PTA7	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
7	PortPin_7	8	8	PTA8	☒	FCUART2_RX	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
8	PortPin_8	9	9	PTA9	☒	FCUART2_TX	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
9	PortPin_9	10	10	PTA10	☒	JTAG_TDO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	
10	KEY3	11	11	PTA11	☒	GPIO	PULL_D...	PULL_D...	☒	PORT_P...	PORT	

3) Configure PWM if the transmission source is FTU channels.

PwmChannelConfigSet

Name: PwmChannelConfigSet

PwmChannel

Ind...	Name	PwmC...	PWM ...	PwmFtuChannel	PwmP...	Pwml...	Defaul...	Defaul...	PwmD...
0	RGB1_BLUE	0	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
1	RGB1_GREEN	1	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
2	RGB1_RED	2	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
3	RGB2_BLUE	3	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
4	RGB2_GREEN	4	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
5	RGB2_RED	5	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	10000	1.0E-4	0x0
6	MSC_Pwm_0	6	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
7	MSC_Pwm_1	7	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
8	MSC_Pwm_2	8	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
9	MSC_Pwm_3	9	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
10	MSC_Pwm_4	10	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
11	MSC_Pwm_5	11	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
12	MSC_Pwm_6	12	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000
13	MSC_Pwm_7	13	FTU	/Pwm/Pwm/PwmChannelC...	PWM_...	PWM_L...	1000	0.001	0x4000

4) Configure MSC general configurations.

a) Configure general configuration.

Name: Msc

General | MscEcucPartitionRef | MscChannel | Published Information

Config Variant: VariantPreCompile

MscGeneral

Name: MscGeneral

Msc Development Error Detection: ☒ Provide Msc VersionInfo Api: ☒

Msc MultiCore Support: ☐

Msc Timeout Duration (0 -> 1000000): 10000

Msc Disable Production Error Reporting: ☐ Msc Use Polling: ☒

Msc Dem Event Parameter Refs

Name: MscDemEventParameterRefs

Msc Timeout Dem Error: /Dem/Dem/DemConfigSet/DemEventParameter_MscTimeout

MscGlobalConfig

Name: MscGlobalConfig

b) Configure Msc channel 1 Tx configuration.

Name  MscChannel_1

Channel Config | SRL Selection | SRH Selection | Emergency Load Enable | MscTransSourceSelection

Msc Channel ID  1 

Msc Hw Channel Index  1 

MscChannelEcuPartitionRef  /EcuC/EcuC/EcuPartitionCollection_0/EcuPartition_0

Tx Configuration

Name  TxConfiguration

Msc Passive Frame Length(bit) (1 -> 31)  3 

Msc Passive Frame Number (1 -> 15)  2 

Msc SRH Data Selection Bit  MSC_DATA_SELECTION_BIT_DISABLE 

Msc SRH data length (0 -> 16)  0 

Msc SRL Data Selection Bit  MSC_DATA_SELECTION_BIT_ENABLE 

Msc SRL data length (0 -> 16)  16 

Msc Transmission Polarity  MSC_TX_INVERTED_POLARITY_EN 

Msc Tx Mode  MSC_TX_SYNCHRONOUS 

Msc Transmission Trigger  MSC_TX_TRIGGER_CONTINUOUS 

Msc Channel Tx Frequency (Hz)  1500000 

MscDownstreamClockRef  cu/Mcu/McuModuleConfiguration/MCU_Demo_FOSC24M/McuClockReferencePoint_MSC1 

Rx Configuration

c) Config Rx Configuration, IO Configuration and Interrupt Configuration.

Channel Config | SRL Selection | SRH Selection | Emergency Load Enable | MscTransSourceSelection

Rx Configuration

Name  RxConfiguration

Msc Rx Clock Ratio  128 

Msc Channel Rx baudrate  11718 

Msc Rx Data Size  MSC_RX_12_BIT_LENGTH 

Msc Rx Data Parity  MSC_RX_PARITY_EVEN 

IO Configuration

Name  IOConfiguration

Msc Serialized Input IO selection  SDIO 

Msc Serialized Input IO Polarity  SDI_POLARITY_INDIDENTICAL 

Msc ChipSelect For ENC  MSC_EN_SELECTION_0 

Msc ChipSelect For ENL  MSC_EN_SELECTION_0 

Msc ChipSelect For ENH  MSC_EN_SELECTION_0 

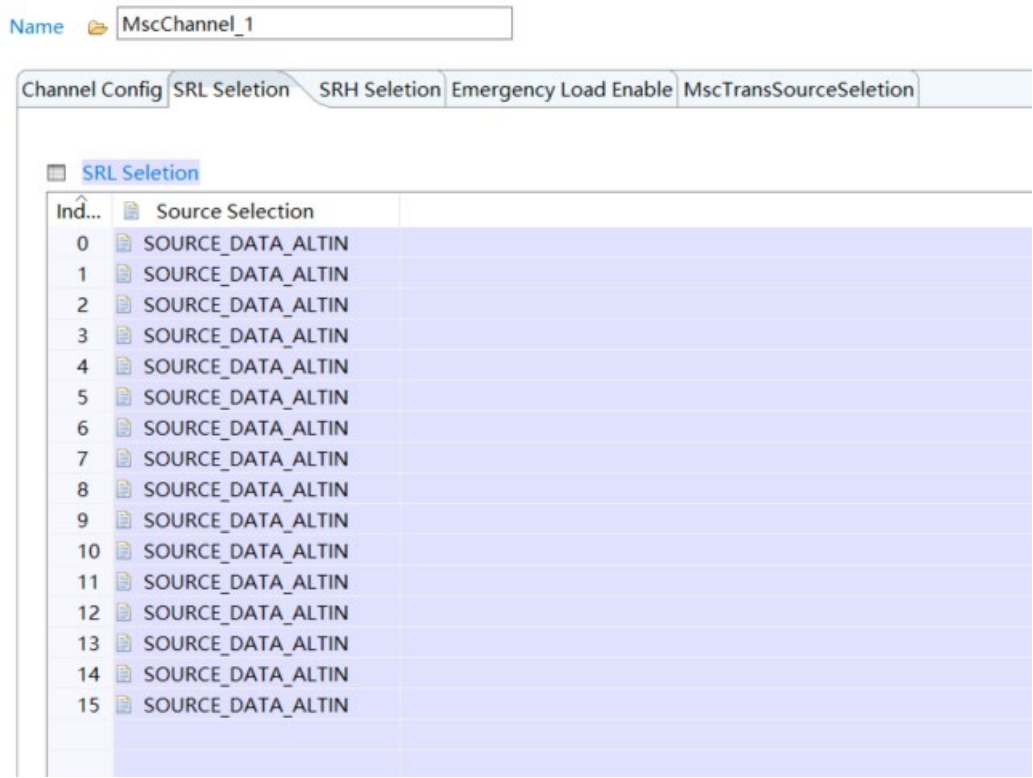
Interrupt Configuration

Name  InterruptConfiguration

Msc Interrupt Delay  ☐ 

Msc Rx Timeout Value (1 -> 65535)  100

d) Choose the SRL source is pwm.



5) Generate Configuration files.